In [1]:
```
import pandas as pd
```

In [3]:
```
df=pd.read_csv("loan_risk_prediction_raw_dataset.csv")
df.head(10)
```

Out[3]:

	Age	Income	LoanAmount	CreditScore	YearsExperience	Gender	Education	City	EmploymentType	LoanApproved
0	56	48353.0	31258.0	675.0	20	Female	High School	Houston	Unemployed	0
1	69	57462.0	23262.0	586.0	6	Male	High School	San Francisco	Self-Employed	0
2	46	44219.0	26530.0	781.0	26	Male	PhD	Houston	Self-Employed	1
3	32	56307.0	11531.0	549.0	11	Male	NaN	New York	Unemployed	0
4	60	37034.0	27871.0	500.0	19	Female	High School	Chicago	Unemployed	0
5	25	47886.0	18106.0	835.0	13	Male	Masters	New York	Salaried	1
6	38	54748.0	25374.0	760.0	9	Female	High School	New York	Self-Employed	1
7	56	NaN	6279.0	599.0	22	Male	PhD	New York	Unemployed	0
8	36	25918.0	25041.0	777.0	29	Female	Bachelors	San Francisco	Unemployed	0
9	40	43415.0	2065.0	382.0	30	Female	High School	San Francisco	Self-Employed	0

1. DATA UNDERSTANDING

In [6]:
```
df.shape
```

Out[6]:
```
(5000, 10)
```

In [8]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Age              5000 non-null   int64
 1   Income           4804 non-null   float64
 2   LoanAmount       5000 non-null   float64
 3   CreditScore      4806 non-null   float64
 4   YearsExperience  5000 non-null   int64
 5   Gender           5000 non-null   object
 6   Education        4802 non-null   object
 7   City             5000 non-null   object
 8   EmploymentType   5000 non-null   object
 9   LoanApproved     5000 non-null   int64
dtypes: float64(3), int64(3), object(4)
memory usage: 390.8+ KB
```

In [10]:
```
df.describe()
```

Out[10]:

	Age	Income	LoanAmount	CreditScore	YearsExperience	LoanApproved
count	5000.000000	4804.000000	5000.000000	4806.000000	5000.000000	5000.000000
mean	43.584600	49738.123022	19870.768600	575.494590	19.599000	0.230200
std	14.919094	15101.361851	8046.542413	160.550839	11.516837	0.421003
min	18.000000	-3731.000000	-10059.000000	300.000000	0.000000	0.000000
25%	31.000000	39608.500000	14455.250000	433.000000	10.000000	0.000000
50%	43.000000	49488.000000	19842.500000	579.000000	20.000000	0.000000
75%	56.000000	59917.000000	25326.750000	712.000000	29.000000	0.000000
max	69.000000	99146.000000	48353.000000	849.000000	39.000000	1.000000

In [12]:
```
df.isnull().sum()
```

Out[12]:
```
Age                 0
Income            196
LoanAmount          0
CreditScore       194
YearsExperience     0
Gender              0
Education         198
City                0
EmploymentType      0
LoanApproved        0
dtype: int64
```

In [14]:
```
df.duplicated().sum()
```

Out[14]:
```
0
```

2. DATA CLEANING (IMPORTANT)

In [25]:
```
# Numerical
df['Income'] = df['Income'].fillna(df['Income'].median())
df.fillna({
    'CreditScore': df['CreditScore'].median(),
    'Education': df['Education'].mode()[0]
}, inplace=True)
```

Recheck

In [28]:
```
df.isnull().sum()
```

Out[28]:
```
Age                0
Income             0
LoanAmount         0
CreditScore        0
YearsExperience    0
Gender             0
Education          0
City               0
EmploymentType     0
LoanApproved       0
dtype: int64
```

Fix Text Columns

In [31]:
```
for col in ['Gender','City','EmploymentType','Education']:
    df[col] = df[col].str.strip().str.title()
```

Check Data Types

In [34]:
```
df.dtypes
```

Out[34]:
```
Age                  int64
Income             float64
LoanAmount         float64
CreditScore        float64
YearsExperience      int64
Gender              object
Education           object
City                object
EmploymentType      object
LoanApproved         int64
dtype: object
```

Ensure: LoanApproved → int Age → int Income → float

Outlier Treatment (Simple & Effective)

In [39]:
```
# Remove extreme values (top 1%)
df = df[df['LoanAmount'] < df['LoanAmount'].quantile(0.99)]
df = df[df['Income'] < df['Income'].quantile(0.99)]
```

3. FEATURE ENGINEERING (VERY IMPORTANT)

Income vs Loan Ratio

In [43]:
```
df['Income_Loan_Ratio'] = df['Income'] / df['LoanAmount']
```

In [47]:
```
print(df.Income_Loan_Ratio)
```

```
0       1.546900
1       2.470209
2       1.666755
3       4.883098
4       1.328765
          ...
4994    2.597970
4995    1.572938
4997    3.187054
4998    4.729088
4999    2.495363
Name: Income_Loan_Ratio, Length: 4900, dtype: float64
```

Risk Level (Core Feature)

In [50]:
```
df['RiskLevel'] = df['CreditScore'].apply(
    lambda x: 'High Risk' if x < 600 else 'Low Risk'
)
```

In [52]:
```
df.RiskLevel.head()
```

Out[52]:
```
0     Low Risk
1    High Risk
2     Low Risk
3    High Risk
4    High Risk
Name: RiskLevel, dtype: object
```

Experience Group

In [55]:
```
df['ExperienceGroup'] = pd.cut(
    df['YearsExperience'],
    bins=[0,2,5,10,20,40],
    labels=['Fresher','Junior','Mid','Senior','Expert']
)
```

In [57]:
```
df.ExperienceGroup.head()
```

Out[57]:
```
0    Senior
1       Mid
2    Expert
3    Senior
4    Senior
Name: ExperienceGroup, dtype: category
Categories (5, object): ['Fresher' < 'Junior' < 'Mid' < 'Senior' < 'Expert']
```

Income Group

In [60]:
```
df['IncomeGroup'] = pd.cut(
    df['Income'],
    bins=[0,30000,60000,100000,200000],
    labels=['Low','Medium','High','Very High']
)
```

In [67]:
```
df.IncomeGroup.head()
```

Out[67]:
```
0    Medium
1    Medium
2    Medium
3    Medium
4    Medium
Name: IncomeGroup, dtype: category
Categories (4, object): ['Low' < 'Medium' < 'High' < 'Very High']
```

In [65]:
```
df.head()
```

Out[65]:

	Age	Income	LoanAmount	CreditScore	YearsExperience	Gender	Education	City	EmploymentType	LoanApproved	Income_Loar
0	56	48353.0	31258.0	675.0	20	Female	High School	Houston	Unemployed	0	1.
1	69	57462.0	23262.0	586.0	6	Male	High School	San Francisco	Self-Employed	0	2.
2	46	44219.0	26530.0	781.0	26	Male	Phd	Houston	Self-Employed	1	1.
3	32	56307.0	11531.0	549.0	11	Male	Bachelors	New York	Unemployed	0	4.
4	60	37034.0	27871.0	500.0	19	Female	High School	Chicago	Unemployed	0	1.

4. EXPLORATORY DATA ANALYSIS (EDA)

Target Distribution

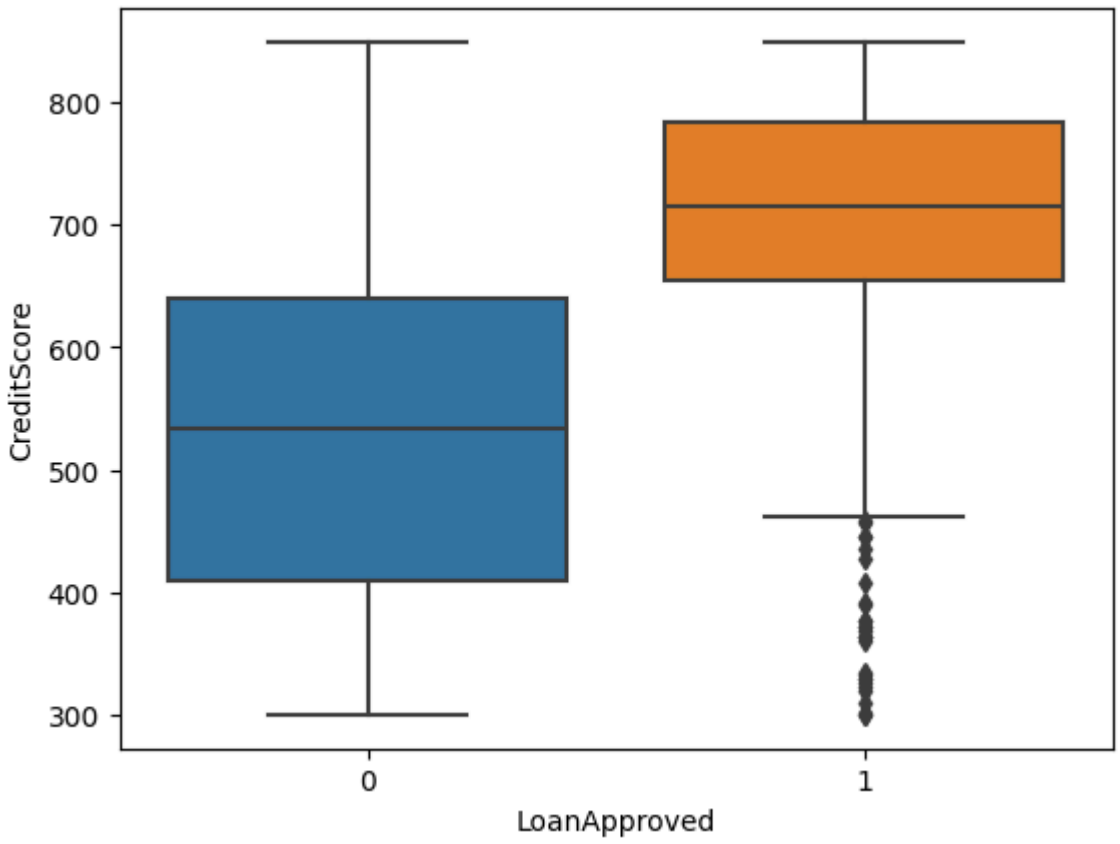
In [71]:
```
df['LoanApproved'].value_counts(normalize=True)
```

Out[71]:
```
LoanApproved
0    0.772449
1    0.227551
Name: proportion, dtype: float64
```

Approval vs Credit Score

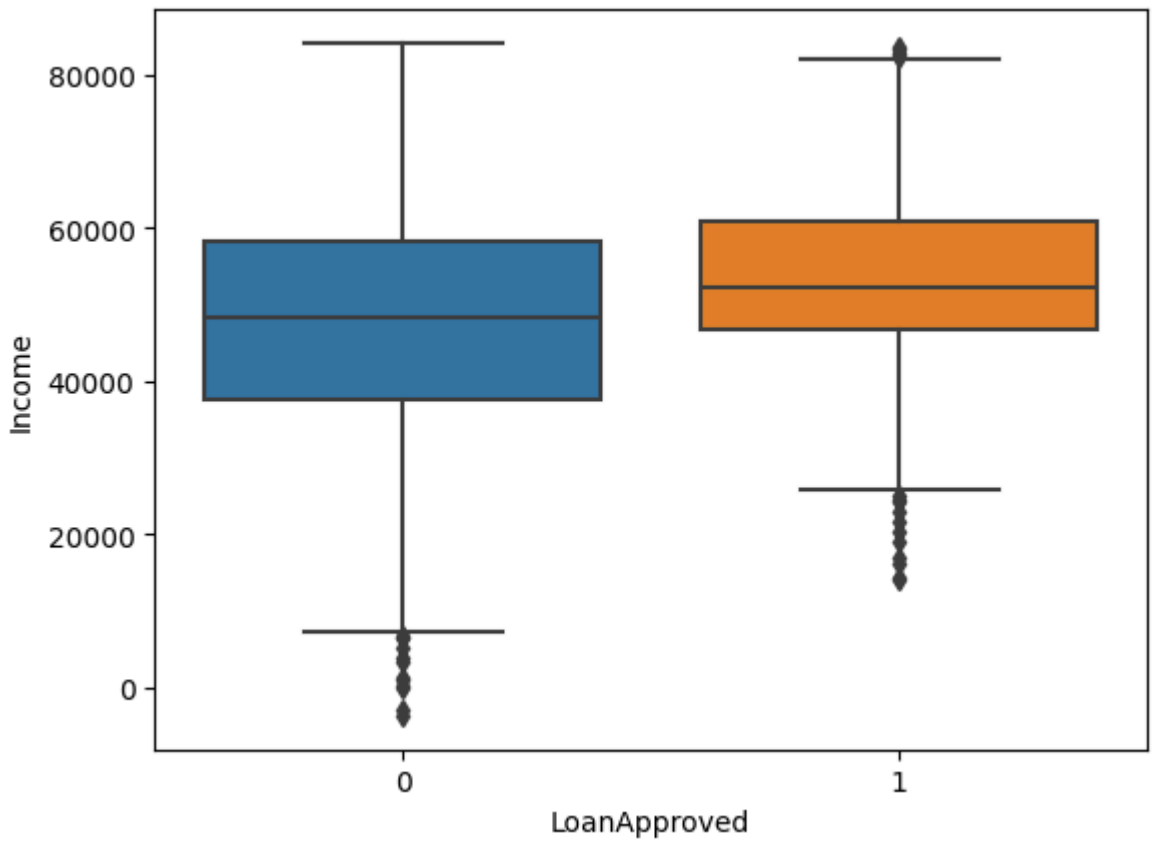
In [74]:
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x='LoanApproved', y='CreditScore', data=df)
plt.show()
```



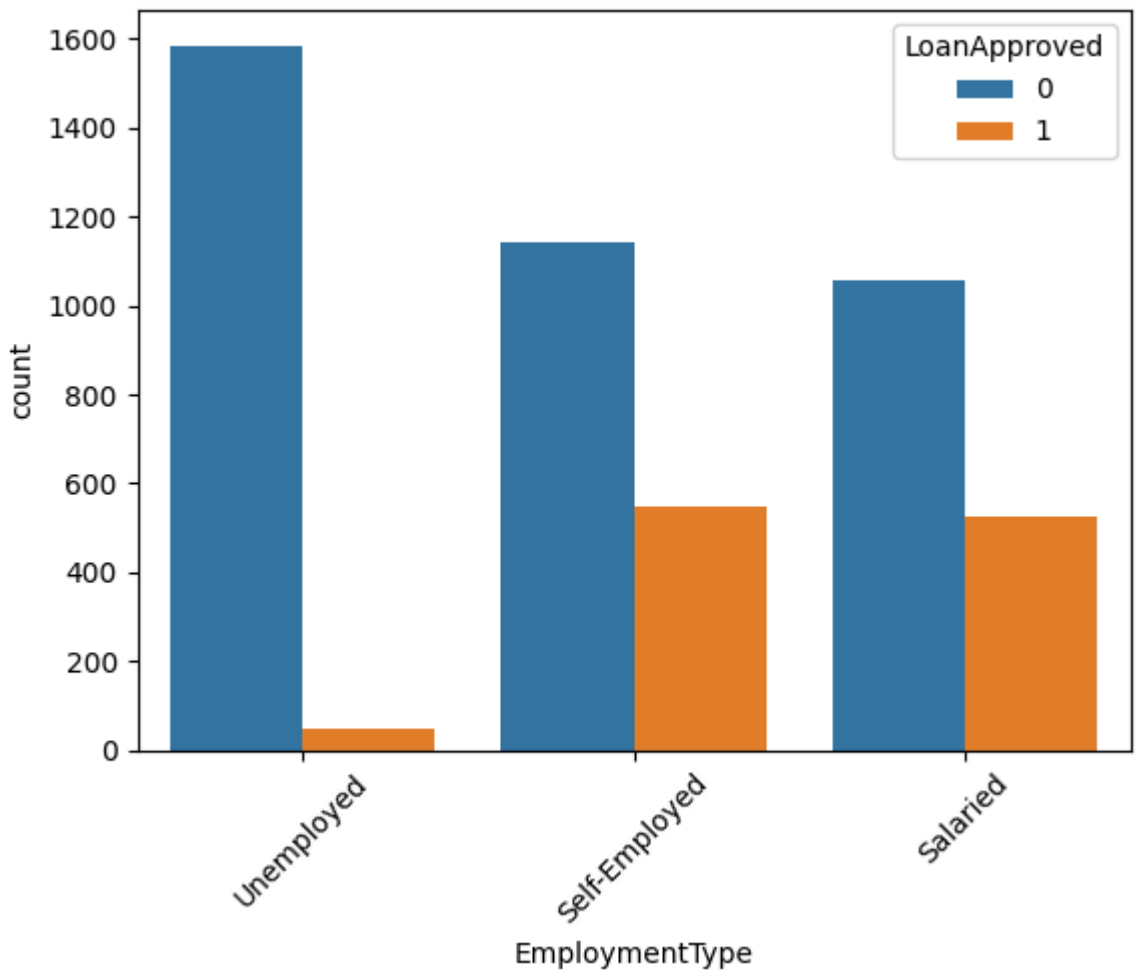
Income vs Approval

In [77]:
```
sns.boxplot(x='LoanApproved', y='Income', data=df)
plt.show()
```



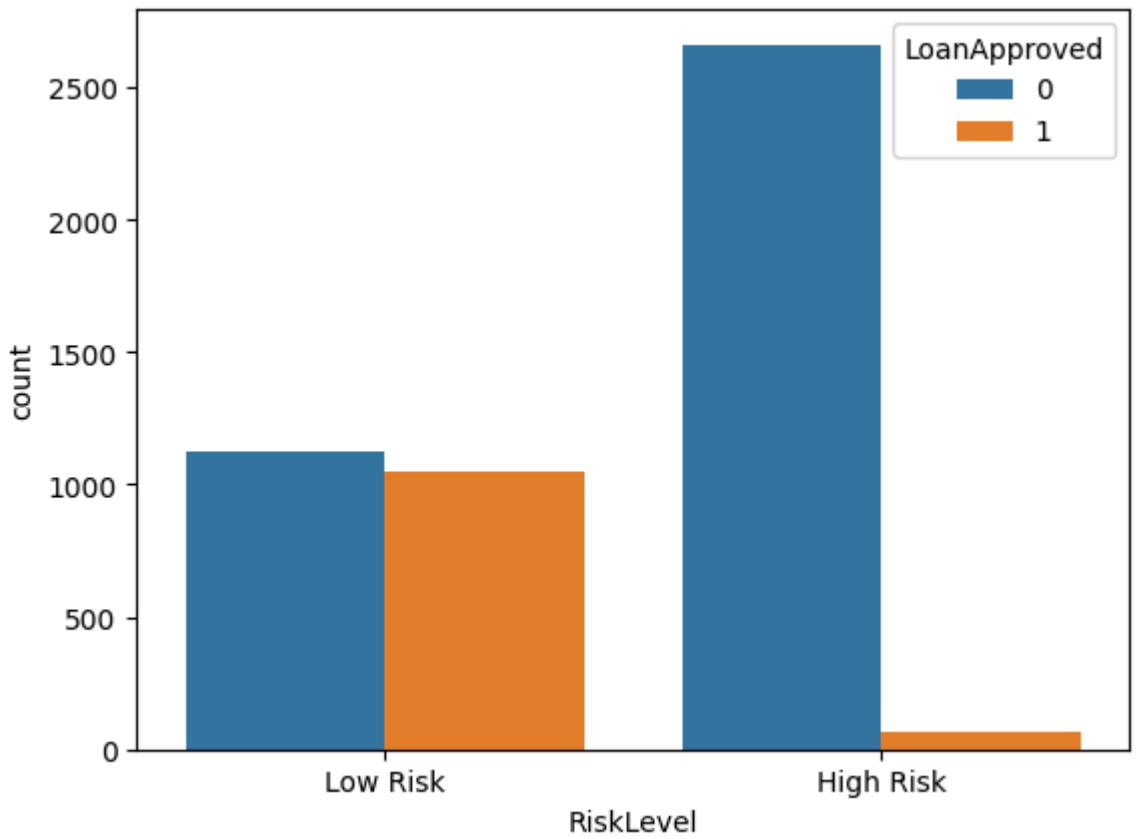
Employment vs Approval

In [80]:
```
sns.countplot(x='EmploymentType', hue='LoanApproved', data=df)
plt.xticks(rotation=45)
plt.show()
```



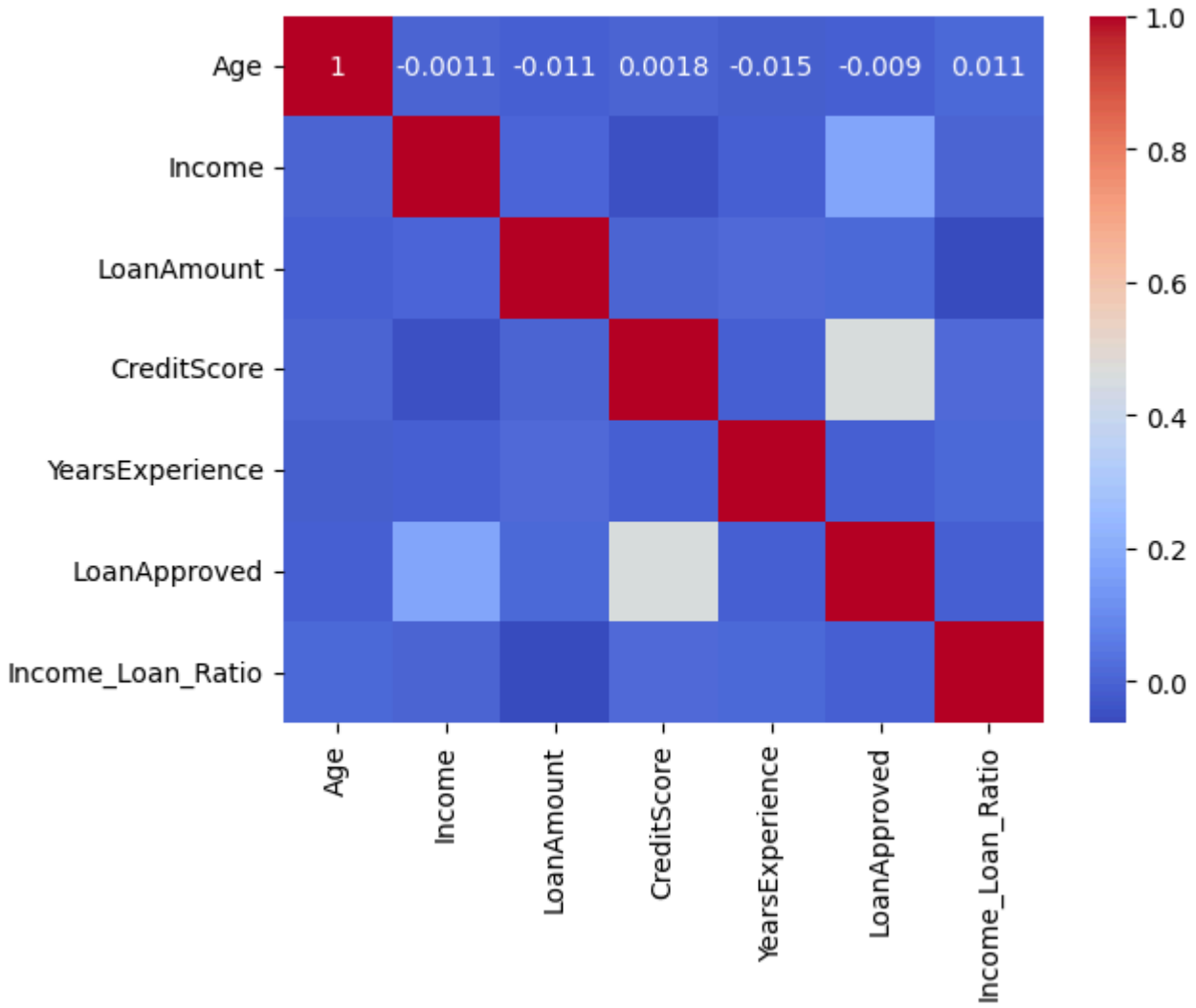
Risk Level vs Approval

In [83]:
```
sns.countplot(x='RiskLevel', hue='LoanApproved', data=df)
plt.show()
```



Correlation

In [86]:
```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```



6. FINAL DATA PREPARATION (FOR SQL)

In [89]:
```
df.columns = df.columns.str.lower()
```

Final Check

In [92]:
```
df.isnull().sum()
df.dtypes
```

Out[92]:
```
age                    int64
income               float64
loanamount           float64
creditscore          float64
yearsexperience        int64
gender                object
education             object
city                  object
employmenttype        object
loanapproved           int64
income_loan_ratio    float64
risklevel             object
experiencegroup     category
incomegroup         category
dtype: object
```

Export Clean Dataset

In [95]:
```
df.to_csv("cleaned_credit_data.csv", index=False)
```

In [99]:
```
import pandas as pd

df = pd.read_csv("cleaned_credit_data.csv", quotechar='"')
print(df.head())
```

```
   age   income  loanamount  creditscore  yearsexperience  gender  \
0   56  48353.0     31258.0        675.0               20  Female
1   69  57462.0     23262.0        586.0                6    Male
2   46  44219.0     26530.0        781.0               26    Male
3   32  56307.0     11531.0        549.0               11    Male
4   60  37034.0     27871.0        500.0               19  Female

     education           city employmenttype  loanapproved  income_loan_ratio  \
0  High School        Houston     Unemployed             0           1.546900
1  High School  San Francisco  Self-Employed             0           2.470209
2          Phd        Houston  Self-Employed             1           1.666755
3    Bachelors       New York     Unemployed             0           4.883098
4  High School        Chicago     Unemployed             0           1.328765

   risklevel experiencegroup incomegroup
0   Low Risk          Senior      Medium
1  High Risk             Mid      Medium
2   Low Risk          Expert      Medium
3  High Risk          Senior      Medium
4  High Risk          Senior      Medium
```

In [ ]:

In [109…

```python
import pandas as pd
from sqlalchemy import create_engine

# Database credentials
db_user = "postgres"
db_password = "soja1014"
db_host = "localhost"
db_port = "5432"
db_name = "CreditRiskProject"

# Create engine with your actual credentials
engine = create_engine(f"postgresql+psycopg2://{db_user}:{db_password}@{db_host}:{db_port}/{db_name}")
# Read CSV with proper quoting
df = pd.read_csv("cleaned_credit_data.csv", quotechar='"')

# Import into PostgreSQL
df.to_sql('credit_data', engine, if_exists='append', index=False)

print("CSV imported successfully!")
```

```
CSV imported successfully!
```

In [ ]: